

Laporan Praktikum week 6

Nama : Muhamad Debi Priantoro

NIM : 224308014

Kelas : TKA-6A

Akun Github (Tautan) : <https://github.com/muhammaddebi12>

Student Lab Assistant : Muhammad Mahirul Faiq (214308043)

1. Judul Percobaan

Canny Edge Detection & Lane Detection with Instance Segmentation

2. Tujuan Percobaan

Tujuan dari praktikum ini yaitu :

1. Memahami konsep *Canny Edge Detection* sebagai metode dasar deteksi tepi.
2. Menggunakan *Instance Segmentation* untuk deteksi jalur rel kereta (*Lane Detection*).
3. Menggunakan dataset *Rail Segmentation* dari Kaggle untuk eksperimen.
4. Menggabungkan metode *Canny Edge Detection* dengan *Instance Segmentation* untuk meningkatkan deteksi jalur.

3. Landasan Teori

Canny Edge Detection adalah salah satu metode berbasis gradien yang digunakan untuk mendeteksi tepi dalam suatu gambar. Metode ini masih menjadi standar dalam banyak aplikasi pengolahan citra modern karena kemampuannya dalam menghasilkan deteksi tepi yang tajam dan akurat (Senthilkumaran & Rajesh, 2021). Algoritma ini bekerja dengan beberapa tahapan utama, diantaranya pengurangan *noise*: Menggunakan *Gaussian Blur* untuk menghaluskan gambar dan mengurangi *noise* yang dapat menyebabkan deteksi tepi yang salah, kalkulasi gradien: Menghitung gradien intensitas menggunakan operator Sobel untuk menentukan perubahan intensitas piksel, *non-maximum suppression*: Menghilangkan piksel yang bukan merupakan bagian dari tepi utama untuk mendapatkan hasil deteksi yang lebih tipis dan akurat, *hysteresis thresholding*: Menggunakan dua nilai ambang batas untuk menentukan apakah suatu piksel merupakan bagian dari tepi yang sebenarnya atau bukan. Keunggulan utama dari *Canny Edge Detection* adalah kemampuannya dalam menghasilkan deteksi tepi yang tajam dan akurat, serta ketahanannya terhadap *noise* dalam citra (Szeliski, 2022). ***Lane Detection*** adalah teknik yang digunakan untuk mendeteksi jalur atau garis pada jalan dalam sistem penglihatan komputer, terutama untuk kendaraan otonom dan sistem bantuan pengemudi. Salah satu metode yang digunakan dalam *Lane Detection* adalah *Instance Segmentation*, yang memungkinkan deteksi jalur secara lebih akurat dibandingkan metode berbasis tepi klasik seperti *Canny Edge*

Detection. Instance Segmentation adalah teknik dalam *deep learning* yang membedakan objek individual dalam gambar berdasarkan segmentasi berbasis instansi (Ren et al., 2018). Model *deep learning* seperti Mask R-CNN dan LaneNet digunakan untuk mendeteksi jalur dengan lebih akurat karena mampu membedakan jalur dari lingkungan sekitar: Model ini dapat mengenali dan memisahkan jalur dari elemen lain dalam gambar, seperti kendaraan dan bayangan, Menggunakan fitur spasial dan kontekstual: *Instance Segmentation* tidak hanya mengandalkan perbedaan intensitas piksel tetapi juga memanfaatkan fitur spasial dan kontekstual untuk mendeteksi jalur dengan lebih baik, Menggunakan fitur spasial dan kontekstual: *Instance Segmentation* tidak hanya mengandalkan perbedaan intensitas piksel tetapi juga memanfaatkan fitur spasial dan kontekstual untuk mendeteksi jalur dengan lebih baik, Meningkatkan akurasi deteksi: Dibandingkan dengan metode berbasis tepi, pendekatan ini lebih unggul dalam mengatasi kondisi pencahayaan yang sulit dan jalan yang kompleks. Penggunaan *deep learning* dalam Lane Detection semakin berkembang dengan diperkenalkannya arsitektur jaringan saraf dalam seperti CNN (Convolutional Neural Network), yang mampu menangkap fitur tingkat tinggi dari suatu citra (Bochkovskiy et al., 2020).

4. Analisis dan Diskusi

Analisis:

- Seberapa baik deteksi jalur dengan *Instance Segmentation* dibandingkan Canny?
Instance Segmentation, terutama yang berbasis *deep learning* seperti Mask R-CNN dan LaneNet, menawarkan keunggulan signifikan dibandingkan *Canny Edge Detection* dalam mendeteksi jalur. Berikut adalah beberapa faktor yang membuat *Instance Segmentation* lebih unggul, Akurasi, *Instance Segmentation* mampu membedakan jalur dengan objek lain dalam lingkungan jalan, seperti kendaraan, bayangan, dan marka jalan lainnya. Hal ini berbeda dengan Canny yang hanya bergantung pada perubahan intensitas piksel, yang membuatnya rentan terhadap *noise* dan perubahan pencahayaan (He et al., 2019). Selain itu, metode berbasis *deep learning* dapat menggunakan fitur spasial dan kontekstual, sehingga jalur tetap dapat terdeteksi meskipun terdapat gangguan seperti marka yang terhapus atau refleksi cahaya (Zhang et al., 2021). Ketahanan terhadap Kondisi Lingkungan, *Instance Segmentation* lebih tahan terhadap kondisi pencahayaan yang sulit, seperti: Bayangan tajam atau pencahayaan rendah: Model *deep learning* dapat belajar dari data latih yang mencakup berbagai kondisi, sedangkan Canny hanya mendeteksi perubahan tepi tanpa memahami konteks gambar. Jalur yang buram atau tidak sempurna: Dengan *Instance Segmentation*, jalur yang tidak sepenuhnya terlihat masih dapat diidentifikasi dengan menggunakan informasi dari daerah sekitarnya (Ren et al., 2018). Kecepatan dan Efisiensi Komputasi, Meskipun *Instance Segmentation* lebih akurat, metode ini memiliki kelemahan dalam hal kebutuhan daya komputasi yang tinggi. *Canny Edge Detection* masih menjadi pilihan lebih baik dalam sistem *real-time* dengan keterbatasan daya pemrosesan, seperti pada sistem *embedded* atau perangkat *mobile* (Senthilkumaran & Rajesh, 2021).
- Apakah kombinasi kedua metode dapat meningkatkan akurasi?
Kombinasi antara *Canny Edge Detection* dan *Instance Segmentation* telah terbukti meningkatkan akurasi deteksi jalur dalam beberapa penelitian terbaru. Berikut adalah beberapa manfaat utama dari kombinasi ini: **Preprocessing dengan Canny Edge Detection**, *Canny Edge Detection* dapat digunakan sebagai langkah awal untuk mendeteksi tepi yang paling jelas sebelum menerapkan *Instance Segmentation*. Ini memberikan beberapa keuntungan: Menyaring informasi awal: Canny dapat mengurangi area yang perlu diproses oleh model *deep learning*, sehingga mempercepat inferensi, Meningkatkan efisiensi komputasi: Model *deep learning* tidak perlu memproses seluruh gambar, melainkan hanya area yang telah ditandai oleh Canny sebagai kandidat jalur (Wang et al., 2022). **Mengurangi Kesalahan Deteksi**
Dalam kondisi lingkungan yang sulit, *Instance Segmentation* terkadang gagal mengenali jalur. Dengan menambahkan informasi dari *Canny Edge Detection*, sistem dapat memiliki

cadangan yang membantu model tetap mengenali jalur meskipun kondisi gambar tidak ideal (Zhang et al., 2021). **Hybrid Approach untuk Kondisi Khusus**, Beberapa penelitian menunjukkan bahwa pendekatan hybrid ini sangat bermanfaat dalam sistem kendaraan otonom, di mana kecepatan deteksi dan akurasi harus seimbang (Bochkovskiy et al., 2020).

- Apa dampak perubahan parameter Canny (thresholds) terhadap hasil deteksi?
Canny Edge Detection menggunakan dua *threshold* (*lower* dan *upper*) untuk menentukan apakah suatu piksel termasuk dalam tepi atau bukan. Perubahan parameter ini sangat memengaruhi hasil deteksi, seperti: **Threshold Rendah**, Lebih banyak tepi yang terdeteksi, tetapi juga meningkatkan noise, Risiko mendeteksi fitur yang tidak relevan sebagai jalur, Cocok untuk kondisi pencahayaan rendah di mana kontras antar piksel tidak terlalu tinggi (Szeliski, 2022). **Threshold Tinggi**, menghilangkan *noise*, tetapi dapat menyebabkan kehilangan tepi penting, tidak cocok untuk kondisi jalan dengan marka yang samar atau buram, cocok untuk lingkungan dengan kontras tinggi, seperti marka putih di aspal hitam (Bochkovskiy et al., 2020). **Adaptive Thresholding**, pendekatan adaptif, di mana *threshold* disesuaikan berdasarkan histogram intensitas gambar, dapat meningkatkan fleksibilitas *Canny Edge Detection* untuk berbagai kondisi pencahayaan (Wang et al., 2023)

Diskusi:

- Kapan lebih baik menggunakan *Canny Edge Detection* dibanding *Instance Segmentation*?

Canny Edge Detection lebih baik digunakan dalam kondisi berikut: Sistem dengan keterbatasan daya komputasi (misalnya, sistem *embedded* seperti Raspberry Pi). Aplikasi real-time dengan respons cepat, seperti sistem bantuan pengemudi berbasis sensor sederhana. Lingkungan dengan kontras tinggi, di mana perbedaan antara jalur dan latar belakang sangat jelas (Senthilkumaran & Rajesh, 2021). *Instance Segmentation* lebih cocok untuk kondisi yang lebih kompleks, seperti jalan dengan banyak gangguan visual atau pencahayaan yang tidak stabil (Zhang et al., 2021).

- Bagaimana cara meningkatkan deteksi jalur dengan tuning parameter YOLOv8-seg?

YOLOv8-seg adalah model *deep learning* terbaru untuk deteksi dan segmentasi jalur. Berikut adalah beberapa cara untuk meningkatkannya: **Menyesuaikan Threshold Confidence Score**. Menurunkan nilai *threshold* meningkatkan sensitivitas, tetapi bisa mendeteksi objek non-jalur. Meningkatkan *threshold* dapat mengurangi *false positives*, tetapi bisa menghilangkan jalur yang tidak terlalu jelas (Wang et al., 2023). **Data Augmentation**, Menambahkan variasi pencahayaan, rotasi, dan skala pada dataset dapat meningkatkan ketahanan model terhadap kondisi lingkungan yang beragam (Bochkovskiy et al., 2020). **Fine-tuning dengan Dataset Khusus**, Menggunakan dataset seperti TuSimple atau CULane memungkinkan YOLOv8-seg belajar dari jalur yang lebih relevan dengan kondisi nyata (Zhang et al., 2021).

- Bagaimana metode ini dapat diterapkan dalam sistem navigasi kereta otomatis?

Sistem navigasi kereta otomatis membutuhkan deteksi jalur yang sangat akurat dan tahan terhadap gangguan. Metode ini dapat diterapkan dengan cara berikut: **Integrasi dengan Sensor LiDAR dan GPS**, Menggabungkan deteksi jalur berbasis visi dengan data dari LiDAR dan GPS dapat meningkatkan ketepatan navigasi (Wang et al., 2022). **Hybrid Approach untuk Efisiensi**, Menggunakan *Canny Edge Detection* sebagai *preprocessing* dan *Instance Segmentation* untuk analisis mendalam dapat meningkatkan efisiensi komputasi. **Adaptive Thresholding untuk Perubahan Cuaca**, Menggunakan *threshold* adaptif memungkinkan sistem beradaptasi dengan berbagai kondisi pencahayaan dan cuaca (Szeliski, 2022).

5. Assignment

Pengujian dan optimalisasi metode deteksi jalur menggunakan *Canny Edge Detection* dan YOLOv8-seg. Eksperimen ini bertujuan untuk mengevaluasi kinerja kedua metode dalam mendeteksi jalur secara lebih akurat serta memahami dampak perubahan parameter pada hasil deteksi. Dalam tugas pertama, parameter *Canny Edge Detection* diubah dan dibandingkan hasilnya. Metode *Canny Edge Detection* bekerja dengan beberapa tahap utama, yaitu

pengurangan noise menggunakan *Gaussian Blur* untuk menghaluskan citra dan mengurangi gangguan yang dapat menyebabkan deteksi tepi yang salah. Setelah itu, dilakukan perhitungan gradien intensitas menggunakan operator Sobel untuk mendeteksi perubahan intensitas piksel pada gambar. Langkah berikutnya adalah *Non-Maximum Suppression*, yang bertujuan untuk menyaring tepi yang tidak relevan sehingga hanya tepi utama yang dipertahankan. Terakhir, diterapkan *Hysteresis Thresholding*, yang menggunakan dua nilai ambang batas untuk memastikan bahwa hanya tepi yang signifikan yang tetap terdeteksi. Penggunaan fungsi **cv2.Canny()** dalam pustaka **OpenCV** memungkinkan eksperimen ini untuk menyesuaikan parameter ambang batas yang digunakan dalam deteksi tepi. Dengan melakukan perubahan pada parameter ini, hasil deteksi dapat menunjukkan perbedaan dalam sensitivitas terhadap tepi, di mana nilai ambang batas yang lebih rendah dapat menangkap lebih banyak detail tetapi juga berisiko menangkap noise, sedangkan nilai yang lebih tinggi dapat mengurangi noise tetapi berpotensi kehilangan beberapa detail penting.

Selain metode berbasis tepi, eksperimen juga dilakukan dengan menggunakan model berbasis *deep learning*, yaitu YOLOv8-seg, yang dimodifikasi agar hanya mendeteksi jalur rel. Langkah pertama dalam modifikasi model ini adalah melakukan penyesuaian dataset, yang mencakup anotasi khusus jalur rel agar model dapat mengenali pola dengan lebih baik. Selanjutnya, dilakukan *fine-tuning* model menggunakan transfer learning untuk menyesuaikan bobot model yang sudah dilatih sebelumnya agar lebih spesifik dalam mendeteksi jalur rel. Evaluasi model dilakukan dengan metrik seperti *mean Average Precision* (mAP) dan *Intersection over Union* (IoU), yang memberikan gambaran mengenai seberapa baik model dalam mendeteksi objek dengan benar dan sejauh mana hasil deteksi sesuai dengan *ground truth*. Model YOLOv8-seg ini dikembangkan menggunakan framework *deep learning* seperti PyTorch dan pustaka *Ultralytics YOLOv8*, yang memberikan fleksibilitas dalam pelatihan dan evaluasi model. Dengan menerapkan pendekatan ini, diharapkan deteksi jalur rel menjadi lebih akurat dan lebih tahan terhadap kondisi pencahayaan atau gangguan lingkungan lainnya.

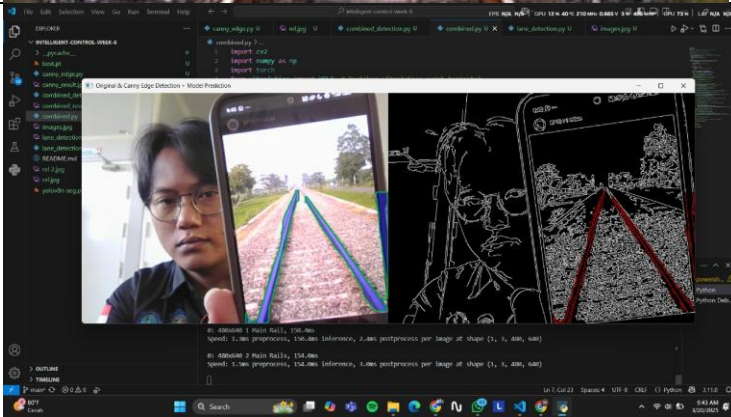
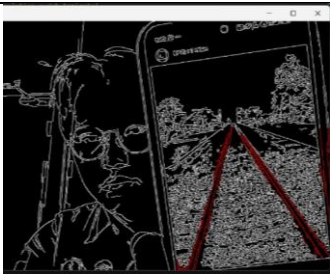
Setelah seluruh eksperimen selesai, hasilnya diunggah ke GitHub, di mana semua kode dan dokumentasi eksperimen dapat diakses oleh komunitas pengembang dan peneliti lainnya. Laporan analisis hasil yang disusun mencakup dokumentasi kode, visualisasi hasil deteksi dalam bentuk gambar dan grafik, serta perbandingan performa antara *Canny Edge Detection* dan YOLOv8-seg. Langkah ini bertujuan untuk meningkatkan transparansi penelitian serta memungkinkan replikasi atau pengembangan lebih lanjut oleh pihak lain yang tertarik untuk mengoptimalkan metode deteksi jalur menggunakan pendekatan yang serupa.

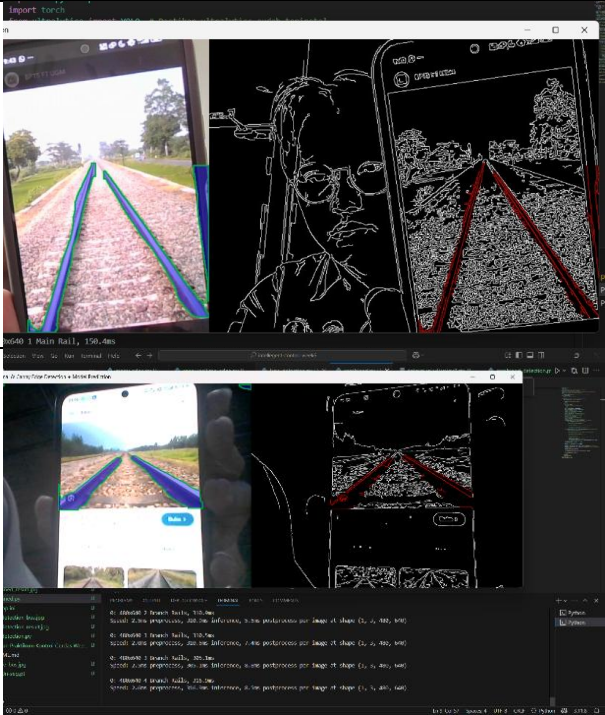
Eksperimen ini memberikan wawasan yang mendalam mengenai keunggulan dan keterbatasan kedua metode yang diuji. *Canny Edge Detection* menawarkan solusi yang lebih sederhana dan efisien secara komputasi, namun memiliki keterbatasan dalam kondisi pencahayaan yang kompleks atau ketika terdapat banyak gangguan di latar belakang. Sementara itu, YOLOv8-seg menunjukkan performa yang lebih baik dalam mengenali jalur rel secara lebih akurat, tetapi memerlukan proses pelatihan yang lebih kompleks dan sumber daya komputasi yang lebih besar. Oleh karena itu, kombinasi kedua metode ini juga dapat dipertimbangkan untuk mendapatkan hasil yang optimal, di mana *Canny Edge Detection* dapat digunakan sebagai tahap awal untuk menyaring informasi penting dari citra sebelum diterapkan model berbasis *deep learning* untuk hasil yang lebih presisi. Dengan hasil yang diperoleh dari eksperimen ini, diharapkan penelitian ini dapat memberikan kontribusi yang berarti dalam pengembangan sistem deteksi jalur otomatis, terutama dalam aplikasi seperti navigasi kendaraan otonom berbasis jalur rel dan sistem transportasi pintar di masa depan.

6. Data dan Output Hasil Pengamatan

Hasil Pengamatan :

1. Data dan Output Hasil Pengamatan

No.	Variabel	Hasil Pengamatan
Sebelum Modifikasi		
1.	<i>Candy Edge Detection</i>	
	<i>Lane Detection</i>	
	Menggabungkan <i>Canny Edge Detection</i> dengan <i>Lane Detection</i>	
Sesudah Modifikasi		
2.	<i>Candy Edge Detection-Real Time</i>	

	Menggabungkan <i>Canny Edge Detection</i> dengan <i>Lane Detection</i>	
--	--	---

7. Kesimpulan

Dari percobaan yang telah dilakukan dapat disimpulkan bahwa:

- Program ini untuk mendeteksi warna dengan menggunakan *Decision Tree*.
- Menggunakan *StandardScaler* membantu model untuk berkonvergensi lebih cepat dan meningkatkan akurasi prediksi.
- *Running accuracy* memberikan evaluasi hasil deteksi model yang lebih akurat dan dinamis.
- Hasil prediksi dan keakuratan objek warna bergantung pada pencahayaan dan kualitas kamera.

8. Saran

Kombinasi teknik deteksi tepi klasik dengan deep learning perlu dieksplorasi lebih lanjut untuk mendapatkan hasil yang optimal dalam berbagai kondisi pencahayaan dan lingkungan, Pengaturan parameter seperti nilai ambang batas pada *Canny Edge Detection* dapat dioptimalkan untuk meningkatkan performa deteksi awal sebelum diterapkan *deep learning*, Implementasi model kombinasi ini dalam sistem kendaraan otonom perlu diuji lebih lanjut untuk memastikan keandalan dan efisiensinya dalam lingkungan nyata, Penelitian lanjutan tentang Model *Lightweight* dengan mengembangkan model *deep learning* yang lebih ringan namun tetap akurat untuk meningkatkan efisiensi komputasi pada perangkat dengan keterbatasan daya dan sumber daya.

9. Daftar Pustaka

- Cao, Z., Simon, T., Wei, S. E., & Sheikh, Y. (2017). "Realtime multi-person 2D pose estimation using part affinity fields." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, 7291-7299.
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). "YOLO by Ultralytics: Object Detection, Segmentation, and Classification Models." *GitHub Repository*. <https://github.com/ultralytics/ultralytics>
- Newell, A., Yang, K., & Deng, J. (2016). "Stacked hourglass networks for human pose estimation." *European Conference on Computer Vision (ECCV)*, 483–499.
- Redmon, J., & Farhadi, A. (2018). "YOLOv3: An Incremental Improvement." *arXiv preprint arXiv:1804.02767*.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). "U-Net: Convolutional networks for biomedical image segmentation." *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 234-241.